

```
--TRASTEUS 6/02/2025 TABLAS [2] // DEPARTAMENTO Y EMPLEADO
--DATOS TABLA:
https://aulavirtual33.educa.madrid.org/ies.piobaroja.madrid/pluginfile.php/150269/mod\_resource/content/1/subconsultas.sql
--EJEMPLOS A PROBAR:
https://aulavirtual33.educa.madrid.org/ies.piobaroja.madrid/pluginfile.php/150105/mod\_label/intro/imagen%20%282%29.png
```

```
/*Trata de entender estos comandos con las tablas departamento y empleado.
Sobretudo, fijate en el movimiento de los datos y que es lo que resulta al
ejecutar el comando*/
```

```
-----
/* DATOS A INSERTAR DEL ENLACE
```

```
CREATE TABLE departamentos (
    id_departamento NUMBER PRIMARY KEY,
    nombre_departamento VARCHAR2(50) NOT NULL
);
```

```
CREATE TABLE empleados (
    id_empleado NUMBER PRIMARY KEY,
    nombre VARCHAR2(50) NOT NULL,
    salario NUMBER(10, 2) NOT NULL,
    id_departamento NUMBER,
    CONSTRAINT fk_departamento FOREIGN KEY (id_departamento) REFERENCES
departamentos(id_departamento)
);
```

```
INSERT INTO departamentos (id_departamento, nombre_departamento) VALUES (1,
'Ventas');
INSERT INTO departamentos (id_departamento, nombre_departamento) VALUES (2,
'Marketing');
INSERT INTO departamentos (id_departamento, nombre_departamento) VALUES (3,
'IT');
```

```
INSERT INTO empleados (id_empleado, nombre, salario, id_departamento) VALUES (1,
'Juan', 3000, 1);
INSERT INTO empleados (id_empleado, nombre, salario, id_departamento) VALUES (2,
'Ana', 2500, 2);
INSERT INTO empleados (id_empleado, nombre, salario, id_departamento) VALUES (3,
'Luis', 2700, 3);
INSERT INTO empleados (id_empleado, nombre, salario, id_departamento) VALUES (4,
'Carmen', 3200, 2);
INSERT INTO empleados (id_empleado, nombre, salario, id_departamento) VALUES (5,
'Pedro', 2800, 1);
INSERT INTO empleados (id_empleado, nombre, salario, id_departamento) VALUES (6,
'Marta', 3300, 3);
```

--Trabajaremos con 2 tablas: empleados y departamentos

-----SELECT-----

```
--weeeee TABLA DEPARTAMENTOS -> 3 ELEMENTOS // columnas -> ||| ID_DEPARTAMENTO |
NOMBRE_DEPARTAMENTO |||
SELECT *
FROM DEPARTAMENTOS;
```

```
--weeeee TABLA EMPLEADOS -> 6 ELEMENTOS // columnas -> ||| ID_EMPLEADO | NOMBRE
| SALARIO | ID_DEPARTAMENTO |||
SELECT *
FROM EMPLEADOS;
```

```
--weeeeeenooo TABLA DEPARTAMENTOS y EMPLEADOS -> 18 ELEMENTOS
-- // columnas -> ||| ID_DEPARTAMENTO | NOMBRE_DEPARTAMENTO | ID_EMPLEADO |
NOMBRE | SALARIO | ID_DEPARTAMENTO |||
```

```
SELECT *
FROM DEPARTAMENTOS , EMPLEADOS;
```

```
/*Si nos fijamos en las columnas, vemos que nos salen
todas las de la tabla departamento y todas las de la tabla empleados.--> LOS
DATOS SE ESTAN MULTIPLICANDO
Vemos todas las posibles relaciones que podrian ser con los datos existentes.
ESTO YA SE CONSIDERA = PRODUCTO
(estos resultados estan DESFORMALIZADOS, id departamento que es la clave
foranea, que relaciona las 2 tablas
no se ha especificado que son lo mismo.)*/*
```

— — — — —

-----SELECCION/PROYECCION-----

```
--Partamos de esta select
```

```
SELECT NOMBRE_DEPARTAMENTO
FROM empleados, departamentos ;
```

```
/*venimos de la select anterior de producto, vemos 18 elementos,
todas las posibles relaciones existentes o no, con los datos iniciales
Vemos lo mismo que en la tabla anterior,
pero solo la columna especifica -> NOMBRE DEPARTAMENTO */
```

```
--""WHERE(de la tabla empleados, ID_DEPARTAMENTO) SEA = (de la tabla
departamentos, ID DEPARTAMENTO)""
```

```
departamentos, id_departamento);
/*aquí, añadiendo el where, descartamos otras opciones
y cuando comparamos ELEMENTOS IGUALES de DIFERENTES TABLAS estamos
relacionando tablas -> ELEMENTO1 = ELEMENTO2 --> CLAVE FORANEA*/
```



```
--__--__--__--__--__--__--__--__--__--__-JOIN(JOIN)-__--__--__--__--__--__--
-->>>>>>>>>>>>>>>>>>>> // Recordemos este select:[*1] ///
/*Aquí veíamos una tabla de TODAS
las posibles relaciones "REALES" de los datos que teníamos,
señalando la relacion entre tablas -> la clave foranea*/

SELECT *
FROM empleados e, departamentos d
WHERE e.ID_DEPARTAMENTO = d.ID_DEPARTAMENTO;
-- veremos que nos da el mismo resultado que el select a explicar en este
apartado
-->>>>>>>>>>>>>>>>>>>> //
----- select con JOIN -----
/*El JOIN nos sirve para referirnos en una sola linea
a una relacion entre tablas, (claves foraneas) // Debemos tener en cuenta:
- En el apartado anterior, en el FROM pondremos todas las tablas a relacionar(en
este caso trabajamos con 2 tablas)
Con el JOIN solo pondremos una tabla(la tabla que se relacionara con las
escritas en los JOIN)
- Dentro del JOIN escribiremos la tabla a relacionar con la tabla inicial,
(tabla inicial -> en el FROM--> en este ejemplo "EMPLEADOS")
- Dentro del JOIN, para referirnos a la relacion entre tablas ([elemento1 =
elemento2] --> CLAVE FORANEA)
lo haremos con un ON ---> JOIN "tabla a relacionar" ON (CLAVE FORANEA)*/

SELECT *
FROM empleados
JOIN departamentos ON empleados.ID_DEPARTAMENTO = departamentos.ID_DEPARTAMENTO;

/*Para verlo claro lo escribo asi, pero recoremos que asi escribimos menos ;d
:*/

SELECT *
FROM empleados e
JOIN departamentos d ON e.ID_DEPARTAMENTO = d.ID_DEPARTAMENTO;

--__--__--__--__--__--__--__--__--__--

--__--__--__--__--__--__--__--__--__--__-JOIN(JOIN-WHERE)
JOIN/SELECCION-__--__--__--__--__--__--

--Añadiremos un where a la query anterior
/*redordemos que añadiendo el where, descartamos otras opciones
- Sin el where, lo que se pide ver es una tabla con todos los datos
de las 2 tablas (empleado y departamento), pero teniendo en cuenta
la relacion entre las tablas --> e.ID_DEPARTAMENTO = d.ID_DEPARTAMENTO -> Clave
Foranea
- Ahora queremos ver especificamente los datos en los que coincida
que elID_DEPARTAMENTO, que corresponde a la table de empleados sea Mayor que 2
(da igual si te refieres al ID ID_DEPARTAMENTO de empleados o departamento; sus
valores son iguales)*/
```

